

Jorge Barrera, Luke Lapinski, Tanner Tompkins

Reyan Ahmed

CSC 337

May 5, 2026

ThreadLine Project Report

Motivation

For our final project, our group decided to create an e-commerce web application called ThreadLine. The purpose of the project is to simulate a real online clothing store where users can browse products, utilize a shopping cart, and complete a checkout process. We chose this idea because online shopping platforms are very common in the real world and provide a good opportunity to combine both frontend and backend web programming concepts into one application. Another reason we selected this project was because it could support multiple interconnected modules, database management, user authentication, and session handling. By building an online store, we were able to practice working with users, products, and order data together in one complete system. In addition, our group wanted experience using MongoDB with Node.js and Express in a larger project structure. The project allowed us to better understand how routes, databases, sessions, and frontend pages communicate with each other in a full stack application.

Modules Overview

The ThreadLine application is divided into three main modules: the User Module, Product Module, and Order Module. These modules are interconnected and work together to support the workflows of the application.

User Module:

The User Module handles account creation, login, logout, and session management. Users can register with a username and password, then log into the application to access the store. Once a user successfully logs in, a session is created using Express sessions, allowing the website to remember the user while navigating between pages. This module is important because the Product and Order modules depend on authentication. Users must be logged in before they can browse products, add items to the cart, or access checkout pages.

Product Module:

The Product Module manages the shirt inventory displayed on the website. Product information is stored in MongoDB and includes attributes such as the product name, description, category, and price. When users open the products page, the frontend sends requests to the backend to retrieve the available products from the database. The products are then dynamically displayed on the webpage using JavaScript. Users can browse the products and add selected items to their shopping cart. The Product Module interacts closely with the Order Module because products selected by the user are transferred into the cart system.

Order Module:

The Order Module manages the shopping cart and checkout process. When a user clicks the “Add to Cart” button, the selected product is added to the user’s cart. Users can also remove individual items, clear the entire cart, and proceed to checkout. During checkout, the application displays a simulated payment page with prefilled demo payment information. After checkout is completed, the order is finalized and the cart is cleared. This module connects both the User Module and Product Module because it depends on authenticated users and selected products from the inventory database. These modules interact through backend API routes and a shared data flow. An example of this would be the Product Module provides product data that is then accessed by the Order Module when an item is added to a users cart. The User Module enables a session to be properly authenticated before users can access both product and order functionality.

Functionalities

The application supports several important functionalities that simulate the behavior of a real online store.

User Authentication and Session Handling:

Users can create new accounts and log into the application using their credentials. Login sessions remain active while the user navigates through the website, and if a user attempts to access protected pages without being logged in, they are redirected to the login page.

Product Browsing:

After logging in, users can browse the available shirt products on the products page. Product information is loaded dynamically from the MongoDB database using fetch requests between the

frontend and backend. Each product card displays the product name, description, price, product image, and an “Add to Cart” button.

Shopping Cart System:

Users can add products to the shopping cart directly from the products page. The cart page displays all selected items along with the total price of the order. The shopping cart also supports removing individual items, clearing the entire cart, and automatically updating totals.

Checkout Workflow:

The application also includes a simulated checkout system where users can proceed from the cart page to the payment page. Demo payment information is already filled in for testing purposes. When the user places an order, the application processes the request, finalizes the order, clears the shopping cart, and displays a confirmation message. Each workflow in the application follows a pattern where the frontend actions trigger the routes on the backend. An example of this is when a user clicks “add to cart”, the frontend sends a POST request with the productId to the backend. The server retrieves the product from MongoDB, adds the item to the user’s cart, and returns a JSON response. Then, the frontend updates the interface to reflect the response.

Navigation and User Experience:

The website also includes navigation links between the products page, cart page, and logout functionality, creating a smoother user experience and allowing users to move easily between modules.

Technical Details

Frontend Technologies:

The project was developed using a full-stack web development approach with both frontend and backend technologies. The frontend was built using HTML, CSS, and JavaScript. HTML was used to structure the webpages, CSS was used for styling and layout design, and JavaScript was used to handle dynamic page updates and communication with the backend server. The frontend uses fetch requests to send and receive JSON data from backend API routes.

Backend Technologies:

The backend was developed using Node.js and Express.js. Express was used to create the server, manage routes, and process incoming requests from the frontend. The backend is divided into multiple route files, including `userRoutes.js`, `productRoutes.js`, and `orderRoutes.js`. This separation improved organization and made the project easier to manage. The backend routes follow a modular structure to organize user, product, and order route files. Some of the key routes include endpoints for products, adding items to the cart, viewing the cart, and placing orders. The routes are responsible for handling data processing as well as interacting with MongoDB through the [Node.js](#) driver. This specific design allows frontend communication with the backend with clear API endpoints.

Database Structure:

MongoDB Atlas was used as the database system for the project. MongoDB stores data in collections and documents, and the project includes collections for users, products, and orders. MongoDB was connected to the server using the official MongoDB Node.js driver. Each MongoDB collection stores documents relevant to the application. An example of this is that user documents contain a username and password or product documents containing name, description, and price. This allows for efficient retrieval of application data.

Session Handling:

The application uses `express-session` to manage user login sessions. When users log in successfully, session data is stored on the server, and protected pages check whether the user is logged in before allowing access. This session-based authentication system ensures that users remain logged in as they navigate the application. Session data is also used to store the user's shopping cart during their visit. The cart is an array within the session object which allows users to add, remove, and clear their cart without needing to store temporary cart data in the database.

File and Project Organization:

The project files are organized into frontend and backend components, where HTML, CSS, and image files are stored separately from backend route files and server configuration files. Sensitive information, such as the MongoDB connection string and session secret, is stored inside a `.env` file, and the `dotenv` package is used to load these environment variables into the application securely.